

Governance G2 — MAINHEAD Capability Restriction

Implementation report. MAINHEAD becomes an operational-scope label only; user authority resolves exclusively from User + Team + Branch + Organization. Schema unchanged. No data migration. No mobile changes.

STATUS — Implemented and validated. API build, admin typecheck, and admin build all PASS on first attempt. MAINHEAD capability assignment is now restricted to Asset Domains in both the admin UI and the API validator; the effective capability resolver no longer merges MAINHEAD capabilities into a user's authority set. Existing rows are preserved.

1. Files changed

1.1 API

File	Status	Change
apps/api/src/common/canonical-capabilities.ts	edited	Added MAINHEAD_ASSIGNABLE_CAPABILITY_CODES (the 8 Asset Domain codes) and isMainheadAssignableCapabilityCode() predicate. Existing CANONICAL_CAPABILITY_CODES unchanged.
apps/api/src/enterprise/enterprise.service.ts	edited	Imported MAINHEAD_ASSIGNABLE_CAPABILITY_CODES. Added assertMainheadCapabilitiesAllowed() helper that fetches codes for the provided capability IDs and throws BadRequestException with the exact message <i>"MAINHEADs may only be assigned Asset Domain capabilities."</i> plus a disallowedCapabilities array. Wired into syncMainheadCapabilities immediately after assertCapabilitiesExist.
apps/api/src/users/users.service.ts	edited	In getEffectiveCapabilitiesForUser: dropped mainheadIdList declaration, removed the mainheadCapability.findMany from the Promise.all, removed the destructured mainheadCapabilities slot, and removed the for (const row of mainheadCapabilities) merge block. Added an inline comment explaining the change. Preserved the MAINHEAD → branch derivation block (lines 525–590) so BranchCapability inheritance via a MAINHEAD's branch still flows. UserMainheadAccess / UserOperationalRegionAccess visibility logic untouched. The CapabilitySource union literal MAINHEAD is left in place for forward-compatibility with apps/mobile/src/types.ts — simply no longer produced.

1.2 Admin Web

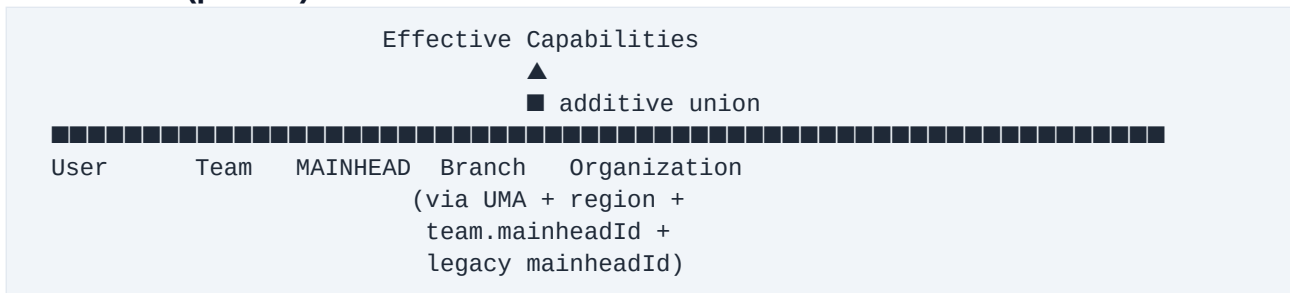
File	Status	Change
apps/admin-web/src/lib/capability-groups.ts	edited	Exported MAINHEAD_PICKER_GROUP_KEYS = ["ASSET_DOMAIN"] as the per-context allow-list.
apps/admin-web/src/components/enterprise-list-client.tsx	edited	CapabilityPicker now accepts an optional groupKeys?: ReadonlyArray<CapabilityGroupKey> prop. When set, the picker also filters the assignable list to capabilities whose capabilityGroupFor (code) is in the allow-list. The MAINHEAD branch of EnterpriseFormModal (line 858) now passes groupKeys={MAINHEAD_PICKER_GROUP_KEYS}. Organization / Branch / Team modal call sites (lines 672, 719, 924) are unchanged.

1.3 Mobile / Schema

- **Mobile:** no changes (as required).
- **Schema / Migrations:** no changes. No schema migration. No data migration. Existing MainheadCapability rows (including any pre-G2 Workspace / Governance assignments) are left in the database; they are no longer consulted by the resolver and no longer editable through the UI.

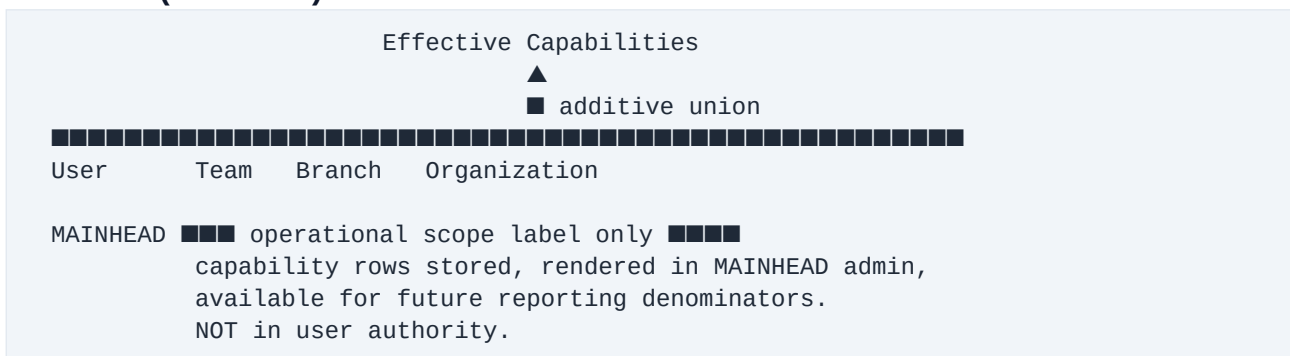
2. Before / after authority model

2.1 Before (pre-G2)



A capability placed on a MAINHEAD was silently promoted into every user with any kind of access to that MAINHEAD. Region access multiplied the widening across every MAINHEAD in the region.

2.2 After (G2 active)



- User authority = User $\cup$ Team $\cup$ Branch $\cup$ Organization.
- MAINHEAD capability rows persist in MainheadCapability but contribute nothing to EffectiveCapability.sources.
- UserMainheadAccess and UserOperationalRegionAccess still control visibility — only authority resolution changed.
- The MAINHEAD $\rightarrow$ branch derivation chain still feeds BranchCapability inheritance (so a user with access to a MAINHEAD continues to inherit the parent branch's capabilities — that path was always Branch-sourced, not MAINHEAD-sourced).

3. Validation results

Step	Command	Result
API build	<code>pnpm --filter @ascure/api exec tsc -p tsconfig.build.json</code>	PASS — exit 0, no diagnostics.
Admin typecheck	<code>pnpm --filter @ascure/admin-web typecheck</code>	PASS — exit 0, no diagnostics.
Admin build	<code>pnpm --filter @ascure/admin-web build</code>	PASS — next build compiled in 5.5s, TypeScript pass in 11.3s, all 21 routes generated.

3.1 Verification checklist (per requirements §6)

Check	How verified	Result
MAINHEAD form only shows Asset Domains	EnterpriseFormModal MAINHEAD branch passes <code>groupKeys={MAINHEAD_PICKER_GROUP_KEYS}</code> to <code>CapabilityPicker</code> ; picker filters via <code>capabilityGroupFor(code) === "ASSET_DOMAIN"</code> .	✓
User form unchanged	<code>UserCapabilityPicker</code> not modified; renders all three groups.	✓
Organization form unchanged	Organization branch <code>CapabilityPicker</code> call site (line 672) has no <code>groupKeys</code> prop.	✓
Team form unchanged	Team branch call site (line 924) has no <code>groupKeys</code> prop.	✓
Branch form unchanged	Branch branch call site (line 719) has no <code>groupKeys</code> prop.	✓
Effective capability resolution still works	User + Team + Branch + Organization queries + merges unchanged. ADMIN override path unchanged. TS + admin build clean.	✓
Workspace access still resolves correctly	<code>Mobile operationalWorkspace.ts</code> reads <code>INSPECTION / MAINTENANCE</code> from the effective set; these now come only from User / Team / Branch / Organization.	✓
MAINHEAD visibility unchanged	<code>UserMainheadAccess</code> queries + <code>directMainheadIds</code> derivation untouched.	✓
Operational Region visibility unchanged	Region → MAINHEAD expansion (lines 545–569) untouched.	✓

4. Backward compatibility review (per requirements §5)

Consumer	Reads MAINHEAD capability?	Impact of G2
Mobile workspace resolver (operationalWorkspace.ts)	Indirectly via <code>getCurrentUserCapabilities</code> ; checks <code>INSPECTION / MAINTENANCE</code> in the effective set.	Intended tightening. Any user who got workspace access only through a MAINHEAD-level grant loses workspace access. See pilot-risk note in §5.
Capability resolver (<code>getEffectiveCapabilitiesForUser</code>)	Used to read <code>mainheadCapability</code> ; no longer does.	Direct change. No MAINHEAD-sourced rows in the effective set.
Template resolution (<code>templates.service.ts</code> , <code>checklist-templates.service.ts</code>)	Reads <code>InspectionTemplate.mainheadId</code> for scope hierarchy. Does NOT read <code>MainheadCapability</code> .	No impact.
Defect governance (<code>defects.service.ts</code>)	No <code>MainheadCapability</code> reads.	No impact.
Inspection workflow (<code>inspections/*.ts</code>)	No <code>MainheadCapability</code> reads.	No impact.
Maintenance workflow (<code>defects</code> → <code>maintenance</code>)	No <code>MainheadCapability</code> reads.	No impact.
Reporting logic	No <code>MainheadCapability</code> reads today. Rows remain available as metadata for future reporting denominators.	No impact.
Admin Web display of effective capabilities	Reads resolver output + renders source tags. MAINHEAD remains a valid <code>CapabilitySource</code> union literal for ABI compatibility.	No display crash. Old cached rows tagged MAINHEAD still render; new responses simply omit the tag.

Confirmed: no current runtime behaviour outside the resolver itself depends on MAINHEAD capability grants. Removing them from the effective set does not break templates, defects, inspections, maintenance, or reporting.

5. Pilot risks discovered

Risk	Likelihood	Mitigation
Existing pilot user(s) granted INSPECTION / MAINTENANCE only at the MAINHEAD level may lose mobile workspace access on first cache refresh after deploy.	Medium	Pre-deploy: enumerate affected users via SQL (<code>MainheadCapability × UserMainheadAccess</code> for workspace codes). For each, grant the same capability at the User or Team level via the existing admin UI so authority survives. After backfill, deploy G2.
Existing <code>MainheadCapability</code> rows with <code>QA_VALIDATION</code> or <code>REPORTING</code> remain in the DB but are no longer UI-editable.	Low	No behavioural effect after G2 and invisible in the MAINHEAD picker. Surface in a one-shot ops report; optional ADMIN-only manual cleanup. Not blocking.
<code>CapabilitySource</code> TypeScript union still includes the MAINHEAD literal in both API and mobile.	Low	Cosmetic. Add a deprecation comment in a follow-up. Removing the literal requires a mobile change, which this sprint disallows.
Direct API attempt to assign INSPECTION to a MAINHEAD returns 400.	Expected	Documented behaviour. Exact message: <i>"MAINHEADs may only be assigned Asset Domain capabilities."</i> Plus a <code>disallowedCapabilities</code> array. Validator runs inside the transaction so partial saves cannot occur.
Drift between the admin UI allow-list (<code>MAINHEAD_PICKER_GROUP_KEYS</code>) and the server allow-list (<code>MAINHEAD_ASSIGNABLE_CAPABILITY_CODES</code>).	Low	Both lists are short and explicit; reviewed together in this sprint. Long-term remedy: persist <code>Capability.kind</code> as a schema column. Flagged for backlog.

6. Deploy order

- **API first.** The resolver change tightens the effective set; the validator rejects bad MAINHEAD writes. Both are server-side truth.
- **Admin Web second.** The MAINHEAD picker restriction prevents admins from attempting Workspace / Governance assignments that the API will now reject anyway.
- **No mobile deploy required.** Mobile is forward-compatible: existing builds continue to call `getCurrentUserCapabilities` and gate workspaces on the returned set, which is now the cleaner one.

Rollback

- Revert the three API files (canonical-capabilities.ts addition, enterprise.service.ts helper + call, users.service.ts resolver edit) and the two admin-web files (capability-groups.ts addition, enterprise-list-client.tsx picker prop + MAINHEAD call site).
- No data side-effects — no rows were deleted, no schema changed. Any backfilled User/Team capability assignments performed during the deploy preparation remain in place and are harmless.